



Pontificia Universidad Javeriana

Facultad de Ingeniería de Sistemas

Asignatura: Bases de Datos

ChocolateDoom

Integrantes:

Juan Diego Rojas Zárate

Nassin Suz Soler

Santiago Baez Parra

Docente:

Oswaldo Calderon Romero

Bogotá D.C., Colombia

Junio 2026

Tabla de Contenido

1	ER Diagram, Relational Schema and Rationale	3
1.1	Entity-Relationship Diagram	3
1.2	Relational Schema	3
1.3	Design Rationale	4
2	DDL and Constraints	4
3	ETL Description and Raw Telemetry Sample	5
3.1	ETL Process	5
3.2	Raw Telemetry Sample	5
4	Queries and Results	5
4.1	Query 1: Average Game Session Duration per Map	6
4.2	Query 2: Trajectory Duration per Player	6
4.3	Query 3: UX Responses for Players with Above-Average Trajectory Duration	7
4.4	Query 4: Most Visited Areas (Hotspots)	8
4.5	Query 5: Average UX Score per Player	10
4.6	Query 6: Total Telemetry Events per Player	10
5	Index Evaluation	11
5.1	Experiment 1: Index on Event Time	11
5.1.1	Performance Analysis	12
5.2	Experiment 2: Index on User Identifier	12
5.2.1	Performance Analysis	12
5.3	Experiment 3: Index on Spatial Coordinates	13
5.3.1	Performance Analysis	13
5.4	Overall Performance Comparison	13
5.5	Conclusions	15
6	Data Dictionary	15
6.1	users	16
6.2	players	16
6.3	episodes	16
6.4	maps	16
6.5	sectors	17
6.6	games	17
6.7	<i>game_{participants}</i>	17
6.8	<i>telemetry_{event}</i>	18
6.9	<i>ux_{instruments}</i>	18

- 6.10 ux_{items} 18
- 6.11 $ux_{responses}$ 19
- 6.12 $ux_{response_{items}}$ 19

1. ER Diagram, Relational Schema and Rationale

1.1 Entity-Relationship Diagram

La Figura 5.4 presenta el modelo Entidad-Relación diseñado para representar la información de telemetría generada por Chocolate Doom y los resultados de los instrumentos de experiencia de usuario (UX).

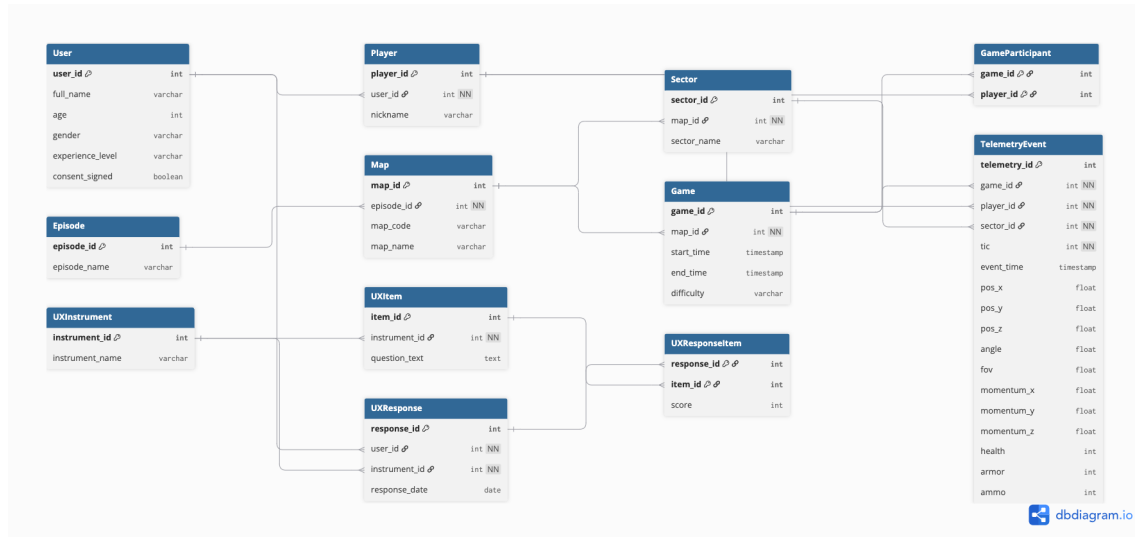


Figura 1: Entity-Relationship Diagram.

1.2 Relational Schema

El modelo relacional derivado del diagrama entidad-relación está compuesto por las siguientes tablas:

- User
- Player
- Episode
- Map
- Sector
- Game
- GameParticipant
- TelemetryEvent
- UXInstrument
- UXItem
- UXResponse
- UXResponseItem

El esquema relacional completo se encuentra disponible en el archivo `relational_schema.md` incluido en el repositorio del proyecto.

1.3 Design Rationale

El modelo fue diseñado para integrar información de telemetría y experiencia de usuario dentro de una única base de datos relacional.

La entidad *User* almacena la información demográfica de los participantes, mientras que *Player* representa la identidad utilizada dentro del videojuego. Esta separación permite mantener independencia entre los datos personales y los datos de juego.

La estructura *Episode* $\rightarrow$ *Map* $\rightarrow$ *Sector* representa la organización natural de los niveles de Doom y facilita futuros análisis espaciales.

La entidad *TelemetryEvent* constituye el núcleo del sistema, almacenando información temporal y espacial generada durante cada tic de juego.

Finalmente, las entidades *UXInstrument*, *UXItem*, *UXResponse* y *UXResponseItem* permiten almacenar diferentes instrumentos UX sin necesidad de modificar el esquema de la base de datos.

2. DDL and Constraints

La implementación física fue realizada utilizando PostgreSQL. Las entidades fueron implementadas mediante tablas relacionales utilizando claves primarias para identificar registros únicos y claves foráneas para garantizar integridad referencial.

Entre las restricciones más importantes se encuentran:

- Claves primarias para todas las entidades principales.
- Claves foráneas para mantener consistencia entre tablas relacionadas.
- Restricciones de unicidad para evitar duplicación de eventos de telemetría.
- Restricciones de dominio para atributos numéricos y temporales.

Un ejemplo de la restricción utilizada para evitar eventos duplicados es:

```
UNIQUE(game_id, player_id, tic)
```

El código SQL completo correspondiente al esquema físico se encuentra incluido en el archivo `schema.sql` del repositorio.

3. ETL Description and Raw Telemetry Sample

3.1 ETL Process

Los datos de telemetría fueron extraídos a partir de archivos TSV generados por una versión modificada de Chocolate Doom. El proceso ETL implementado sigue las siguientes etapas:

1. Extracción de los registros de telemetría desde el archivo TSV original.
2. Carga inicial en la tabla de staging `staging_telemetry`.
3. Conversión de tipos de datos y validación de registros.
4. Inserción de los datos transformados en la tabla `telemetry_event`.

El proceso fue implementado mediante los archivos `staging.sql` y `etl.sql`.

3.2 Raw Telemetry Sample

La Tabla 1 presenta una muestra de la telemetría utilizada durante las pruebas.

Timestamp	Tic	X	Y	Z	Angle	MomX	MomY	Flag
2026-06-03 23:50:56	188	0	-160	0	90.00	-1	1	0
2026-06-03 23:50:57	223	-126	-31	0	80.16	-7	-3	0
2026-06-03 23:50:58	258	-77	-46	0	97.03	9	-1	0
2026-06-03 23:50:59	293	47	-175	0	101.25	-1	2	0
2026-06-03 23:51:00	328	3	106	0	104.06	1	-7	0

Cuadro 1: Sample of raw telemetry data extracted from Chocolate Doom.

4. Queries and Results

Con el fin de analizar la información almacenada en la base de datos, se implementaron seis consultas analíticas basadas en los requerimientos del proyecto. Estas consultas permiten explorar aspectos relacionados con la duración de las sesiones, las trayectorias de los jugadores, los resultados de los instrumentos UX y los patrones de movimiento observados en la telemetría.

4.1 Query 1: Average Game Session Duration per Map

Esta consulta calcula la duración promedio de las sesiones de juego para cada mapa registrado en la base de datos.

Resultado:

```
doom_telemetry=# SELECT
doom_telemetry=#      m.map_code,
doom_telemetry=#      m.map_name,
doom_telemetry=#      ROUND(AVG(EXTRACT(EPOCH FROM (g.
doom_telemetry=# FROM games g
doom_telemetry=# JOIN maps m ON g.map_id=m.map_id
doom_telemetry=# GROUP BY m.map_code,m.map_name
doom_telemetry=# ORDER BY avg_duration_minutes DESC;
 map_code |      map_name      | avg_duration_minutes
-----+-----+-----
  E3M1    | Hell Keep          |          50.00
  E1M1    | Hangar             |          45.00
  E2M1    | Deimos Anomaly     |          40.00
(3 rows)
```

Interpretación:

Los resultados muestran que el mapa Hell Keep (E3M1) presentó la mayor duración promedio de sesión con 50 minutos, seguido por Hangar (E1M1) con 45 minutos y Deimos Anomaly (E2M1) con 40 minutos. Esto indica que algunos mapas requieren más tiempo de exploración o finalización que otros.

4.2 Query 2: Trajectory Duration per Player

Esta consulta calcula la duración de la trayectoria registrada para cada jugador utilizando el rango de tics observados durante la sesión.

Resultado:

```

doom_telemetry=# SELECT
doom_telemetry=#     player_id,
doom_telemetry=#     MIN(tic) AS first_tic,
doom_telemetry=#     MAX(tic) AS last_tic,
doom_telemetry=#     MAX(tic)-MIN(tic) AS trajectory_duration
doom_telemetry=# FROM telemetry_event
doom_telemetry=# GROUP BY player_id;
  player_id | first_tic | last_tic | trajectory_duration
-----+-----+-----+-----
          1 |       188 |    85328 |             85140
(1 row)

```

Figura 2: Entity-Relationship Diagram.

Interpretación:

La consulta muestra que el jugador registrado participó en una trayectoria comprendida entre los tics 188 y 85328, para una duración total de 85140 tics. Este resultado proporciona una medida del tiempo de actividad capturado por la telemetría durante la sesión de juego.

4.3 Query 3: UX Responses for Players with Above-Average Trajectory Duration

Esta consulta identifica los jugadores cuya duración de trayectoria es superior al promedio y recupera las respuestas asociadas a los instrumentos UX completados por dichos participantes.

Resultado:

```

doom_telemetry=# SELECT
doom_telemetry=#     p.nickname,
doom_telemetry=#     ur.response_id,
doom_telemetry=#     ur.response_date
doom_telemetry=# FROM players p
doom_telemetry=# JOIN users u ON p.user_id=u.user_id
doom_telemetry=# JOIN ux_responses ur ON u.user_id=ur.user_id
doom_telemetry=# WHERE p.player_id IN (
doom_telemetry=#     SELECT player_id
doom_telemetry=#     FROM telemetry_event
doom_telemetry=#     GROUP BY player_id
doom_telemetry=#     HAVING MAX(tic)-MIN(tic)>(
doom_telemetry=#         SELECT AVG(duration)
doom_telemetry=#         FROM (
doom_telemetry=#             SELECT MAX(tic)-MIN(tic) AS duration
doom_telemetry=#             FROM telemetry_event
doom_telemetry=#             GROUP BY player_id
doom_telemetry=#         ) t
doom_telemetry=#     )
doom_telemetry=# );
doom_telemetry=#     nickname | response_id | response_date
-----+-----+-----
(0 rows)

```

Figura 3: Entity-Relationship Diagram.

Interpretación:

La consulta no retornó resultados debido a que la base de datos contiene actualmente telemetría de un único jugador. Al no existir múltiples jugadores para comparar, no fue posible identificar participantes con una duración de trayectoria superior al promedio general.

4.4 Query 4: Most Visited Areas (Hotspots)

Esta consulta identifica las zonas más visitadas utilizando agrupaciones espaciales basadas en las coordenadas registradas por la telemetría.

Resultado:

```

doom_telemetry=# SELECT
doom_telemetry-#      FLOOR(pos_x/100)*100 AS zone_x,
doom_telemetry-#      FLOOR(pos_y/100)*100 AS zone_y,
doom_telemetry-#      COUNT(*) AS visits
doom_telemetry-# FROM telemetry_event
doom_telemetry-# GROUP BY zone_x,zone_y
doom_telemetry-# ORDER BY visits DESC
doom_telemetry-# LIMIT 20;
  zone_x | zone_y | visits
-----+-----+-----
    1700 |  -1500 |     17
    1600 |  -1300 |     15
    1500 |  -1300 |     15
    1600 |  -1100 |     15
    1400 |  -1500 |     14
    1700 |  -1200 |     14
    1200 |  -1300 |     13
    1000 |   -600 |     12
     600 |     0  |     11
     900 |   -200 |     11
     600 |   -900 |     11
    1500 |  -1100 |     10
    1400 |  -1400 |     10
    1600 |  -1200 |     10
    1500 |  -1200 |     10
     100 |  -1700 |      9
    1200 |  -1000 |      9
    1200 |   -600 |      9
     400 |   -500 |      9
     900 |   -400 |      9
(20 rows)

```

Figura 4: Entity-Relationship Diagram.

Interpretación:

Los resultados identifican las zonas más frecuentadas durante la sesión de juego. La región ubicada aproximadamente en las coordenadas (1700, -1500) registró el mayor número de visitas, lo que sugiere que dicha área corresponde a una zona de alta actividad o interés dentro del mapa.

4.5 Query 5: Average UX Score per Player

Esta consulta calcula la puntuación promedio obtenida por cada jugador en los instrumentos UX registrados.

Resultado:

```
doom_telemetry=# SELECT
doom_telemetry-#     p.nickname,
doom_telemetry-#     ROUND(AVG(uri.score),2) AS average_score
doom_telemetry-# FROM players p
doom_telemetry-# JOIN users u ON p.user_id=u.user_id
doom_telemetry-# JOIN ux_responses ur ON u.user_id=ur.user_id
doom_telemetry-# JOIN ux_response_items uri ON ur.response_id=uri.
doom_telemetry-# response_id
doom_telemetry-# GROUP BY p.nickname
doom_telemetry-# ORDER BY average_score DESC;
  nickname | average_score
-----+-----
RookieMarine |          4.00
DoomSlayer01 |          4.00
DemonCrusher |          4.00
TechSoldier  |          4.00
CyberMaria   |          4.00
NightHunter  |          4.00
(6 rows)
```

Figura 5: Entity-Relationship Diagram.

Interpretación:

Todos los jugadores presentan una puntuación promedio de 4.0 en los instrumentos UX registrados. Esto indica una valoración consistente entre los participantes utilizados en los datos de prueba cargados en la base de datos.

4.6 Query 6: Total Telemetry Events per Player

Esta consulta contabiliza el número total de eventos de telemetría registrados para cada jugador.

Resultado:

```

doom_telemetry=# SELECT
doom_telemetry=#     p.nickname,
doom_telemetry=#     COUNT(*) AS total_events
doom_telemetry=# FROM telemetry_event te
doom_telemetry=# JOIN players p ON te.player_id=p.player_id
doom_telemetry=# GROUP BY p.nickname
doom_telemetry=# ORDER BY total_events DESC;
 nickname | total_events
-----+-----
DoomSlayer01 |          1792
(1 row)

```

Figura 6: Entity-Relationship Diagram.

Interpretación:

La consulta muestra que el jugador DoomSlayer01 acumuló un total de 1792 eventos de telemetría. Este resultado coincide con la cantidad de registros cargados mediante el proceso ETL y confirma la correcta ingestión de los datos de telemetría.

En conjunto, estas consultas demuestran la capacidad de la base de datos para integrar información de telemetría y experiencia de usuario, permitiendo obtener indicadores relevantes para el análisis del comportamiento de los jugadores.

5. Index Evaluation

Con el objetivo de evaluar el impacto de la indexación sobre el rendimiento de las consultas analíticas, se realizaron pruebas utilizando la herramienta `EXPLAIN ANALYZE` de PostgreSQL.

Para cada índice implementado se seleccionó una consulta representativa del sistema. Cada consulta fue ejecutada cinco veces antes de crear el índice y cinco veces después de su creación. Posteriormente, se calcularon los tiempos promedio de ejecución y se compararon los resultados obtenidos.

5.1 Experiment 1: Index on Event Time

El primer experimento evaluó el siguiente índice:

```

CREATE INDEX idx_telemetry_event_time
ON telemetry_event(event_time);

```

La consulta utilizada fue:

```
SELECT *
FROM telemetry_event
WHERE event_time BETWEEN
'2026-06-03 23:50:56'
AND
'2026-06-03 23:55:00';
```

5.1.1 Performance Analysis

El tiempo promedio de ejecución disminuyó de 0.510 ms a 0.158 ms, representando una mejora aproximada del 69.0 %.

Este resultado demuestra que PostgreSQL puede beneficiarse significativamente de la indexación cuando las consultas realizan filtros sobre atributos temporales.

5.2 Experiment 2: Index on User Identifier

El segundo experimento evaluó el siguiente índice:

```
CREATE INDEX idx_ux_responses_user
ON ux_responses(user_id);
```

La consulta utilizada corresponde a una versión simplificada de la consulta analítica relacionada con las respuestas UX:

```
SELECT
    p.nickname,
    ROUND(AVG(uri.score),2) AS average_score
FROM players p
JOIN users u ON p.user_id=u.user_id
JOIN ux_responses ur ON u.user_id=ur.user_id
JOIN ux_response_items uri ON ur.response_id=uri.response_id
GROUP BY p.nickname
ORDER BY average_score DESC;
```

5.2.1 Performance Analysis

El tiempo promedio de ejecución disminuyó de 0.533 ms a 0.418 ms, representando una mejora aproximada del 21.6 %.

Aunque la mejora fue menor que en el experimento anterior, los resultados muestran que la indexación puede reducir el costo de las operaciones de unión y filtrado relacionadas con los instrumentos UX.

5.3 Experiment 3: Index on Spatial Coordinates

El tercer experimento evaluó el siguiente índice:

```
CREATE INDEX idx_hotspot_position
ON telemetry_event(pos_x, pos_y);
```

La consulta utilizada corresponde al análisis de zonas más visitadas dentro del mapa:

```
SELECT
    FLOOR(pos_x/100)*100 AS zone_x,
    FLOOR(pos_y/100)*100 AS zone_y,
    COUNT(*) AS visits
FROM telemetry_event
GROUP BY zone_x, zone_y
ORDER BY visits DESC;
```

5.3.1 Performance Analysis

El tiempo promedio de ejecución disminuyó de 2.654 ms a 2.596 ms, representando una mejora aproximada del 2.2 %.

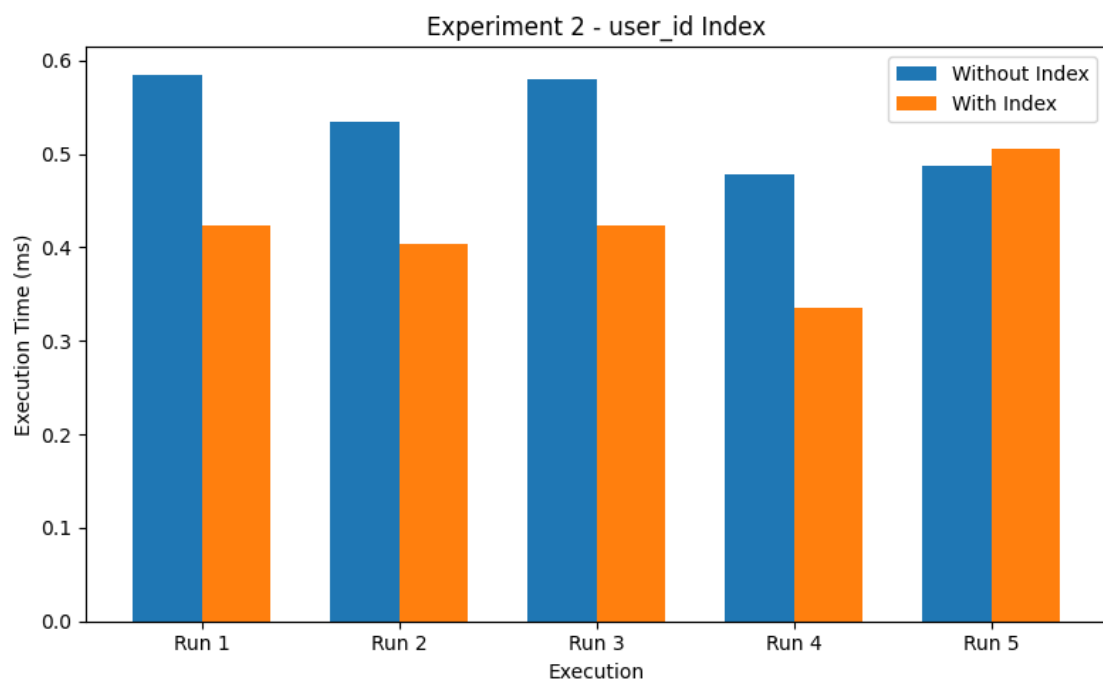
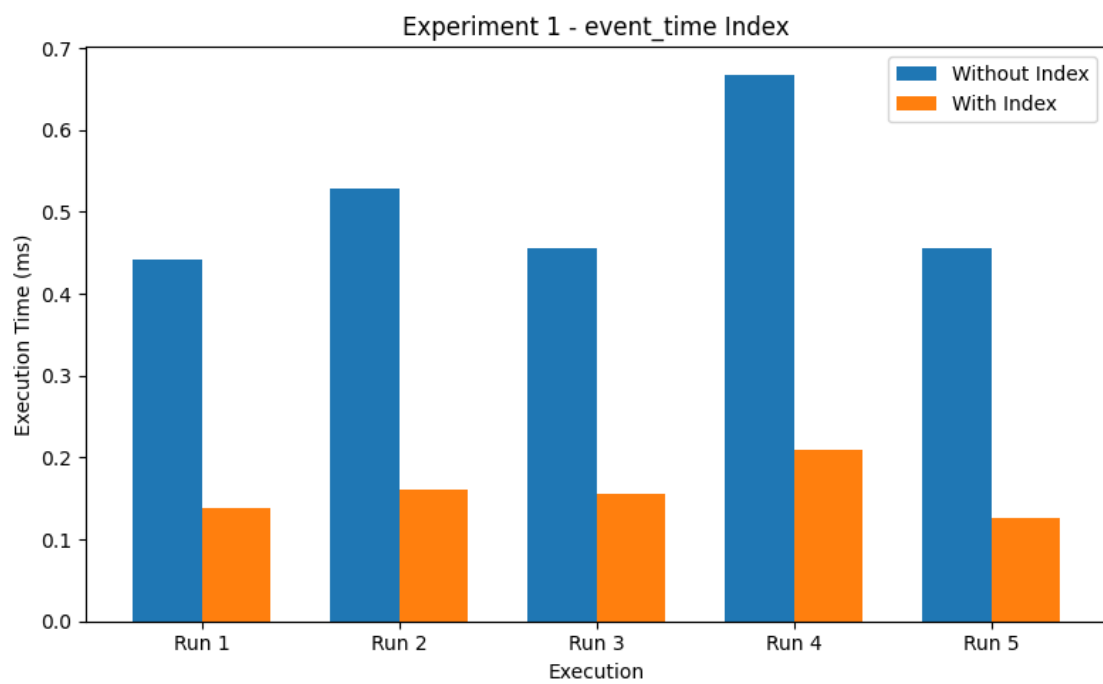
La mejora observada fue limitada debido a que la consulta realiza operaciones de agrupamiento y ordenamiento sobre una gran parte de la tabla. En este tipo de consultas, PostgreSQL continúa necesitando procesar una cantidad considerable de registros incluso cuando existe un índice disponible.

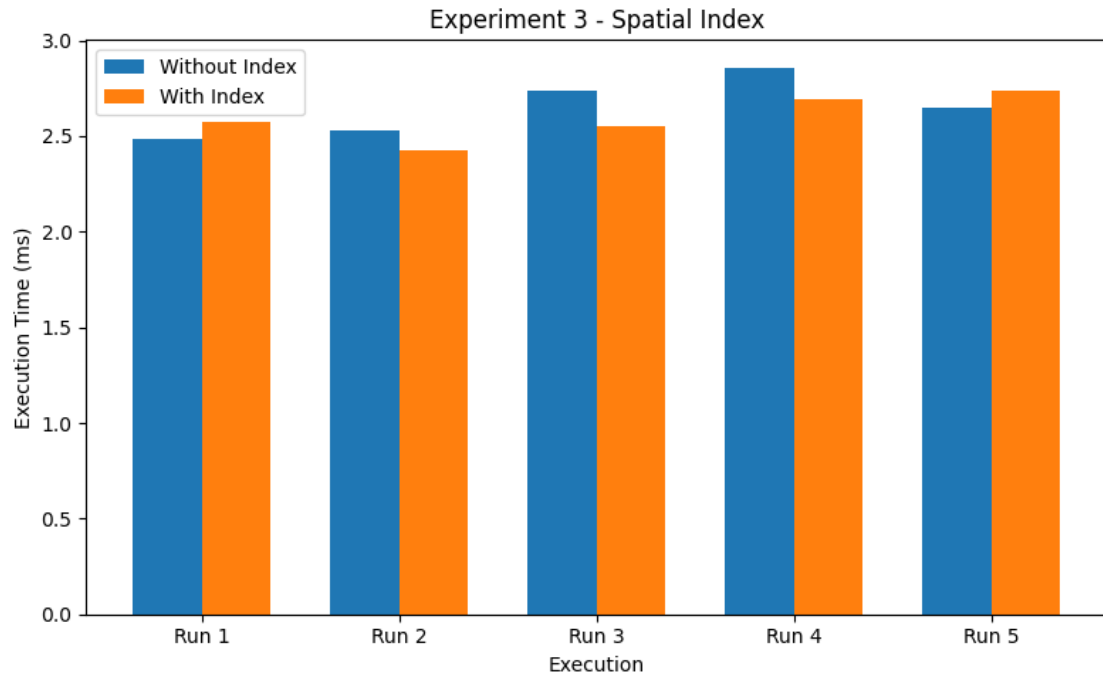
5.4 Overall Performance Comparison

La Tabla 2 resume los tiempos promedio obtenidos en los tres experimentos realizados.

Experiment	Without Index (ms)	With Index (ms)	Improvement (%)
Index 1 (event_time)	0.510	0.158	69.0
Index 2 (user_id)	0.533	0.418	21.6
Index 3 (pos_x,pos_y)	2.654	2.596	2.2

Cuadro 2: Average execution times before and after index creation.





5.5 Conclusions

Los resultados obtenidos demuestran que la utilización de índices puede mejorar significativamente el rendimiento de consultas que realizan filtrado sobre atributos específicos. La mayor mejora se observó en consultas temporales basadas en la columna `event_time`, mientras que las consultas analíticas que involucran agrupamientos y ordenamientos complejos presentaron beneficios más limitados.

En conjunto, los experimentos confirman la importancia de seleccionar índices apropiados de acuerdo con las características de las consultas y del conjunto de datos analizado.

“`latex`

6. Data Dictionary

Esta sección describe las principales entidades y atributos utilizados en la base de datos desarrollada para el análisis de telemetría y experiencia de usuario en Chocolate Doom.

6.1 users

Attribute	Type	Description
$user_id$	SERIAL	Unique identifier of the user
username	VARCHAR(50)	Username
age	INTEGER	User age
gender	VARCHAR(20)	User gender
country	VARCHAR(50)	Country of origin

Cuadro 3: Data dictionary for users.

6.2 players

Attribute	Type	Description
$player_id$	SERIAL	Unique player identifier
$user_id$	INTEGER	Associated user
nickname	VARCHAR(50)	In-game nickname

Cuadro 4: Data dictionary for players.

6.3 episodes

Attribute	Type	Description
$episode_id$	SERIAL	Episode identifier
$episode_{code}$	VARCHAR(10)	Episode code (E1, E2, etc.)
$episode_{name}$	VARCHAR(100)	Episode name

Cuadro 5: Data dictionary for episodes.

6.4 maps

Attribute	Type	Description
map_id	SERIAL	Map identifier
$episode_id$	INTEGER	Parent episode
map_{code}	VARCHAR(10)	Map code (E1M1, E2M1, etc.)
map_{name}	VARCHAR(100)	Map name

Cuadro 6: Data dictionary for maps.

6.5 sectors

Attribute	Type	Description
sector_id	SERIAL	Sector identifier
map_id	INTEGER	Parent map
sector_{name}	VARCHAR(100)	Sector name

Cuadro 7: Data dictionary for sectors.

6.6 games

Attribute	Type	Description
game_id	SERIAL	Game session identifier
map_id	INTEGER	Played map
start_{time}	TIMESTAMP	Session start time
end_{time}	TIMESTAMP	Session end time
difficulty	VARCHAR(50)	Difficulty level

Cuadro 8: Data dictionary for games.

6.7 $\text{game}_p\text{participants}$

Attribute	Type	Description
game_id	INTEGER	Related game
player_id	INTEGER	Participating player

Cuadro 9: Data dictionary for game_participants .

6.8 telemetry_{event}

Attribute	Type	Description
telemetry _i <i>d</i>	SERIAL	Event identifier
game _i <i>d</i>	INTEGER	Associated game
player _i <i>d</i>	INTEGER	Associated player
sector _i <i>d</i>	INTEGER	Associated sector
tic	INTEGER	Game tick
event _t <i>ime</i>	TIMESTAMP	Timestamp of the event
pos _x	DOUBLE PRECISION	X coordinate
pos _y	DOUBLE PRECISION	Y coordinate
pos _z	DOUBLE PRECISION	Z coordinate
angle	DOUBLE PRECISION	Viewing angle
fov	DOUBLE PRECISION	Field of view
momentum _x	DOUBLE PRECISION	X momentum
momentum _y	DOUBLE PRECISION	Y momentum
momentum _z	DOUBLE PRECISION	Z momentum
health	INTEGER	Health value
armor	INTEGER	Armor value
ammo	INTEGER	Ammunition count

Cuadro 10: Data dictionary for telemetry_event.

6.9 ux_{instruments}

Attribute	Type	Description
instrument _i <i>d</i>	SERIAL	Instrument identifier
name	VARCHAR(100)	Instrument name
description	TEXT	Instrument description

Cuadro 11: Data dictionary for ux_instruments.

6.10 ux_{items}

Attribute	Type	Description
item _i <i>d</i>	SERIAL	Question identifier
instrument _i <i>d</i>	INTEGER	Parent instrument
question _t <i>ext</i>	TEXT	Question text

Cuadro 12: Data dictionary for ux_items.

6.11 $ux_{responses}$

Attribute	Type	Description
$response_id$	SERIAL	Response identifier
$instrument_id$	INTEGER	Related instrument
$user_id$	INTEGER	Responding user
$response_date$	TIMESTAMP	Submission date

Cuadro 13: Data dictionary for $ux_{responses}$.

6.12 $ux_{response_items}$

Attribute	Type	Description
$response_id$	INTEGER	Parent response
$item_id$	INTEGER	Related question
$score$	INTEGER	Assigned score

Cuadro 14: Data dictionary for $ux_{response_items}$.